EXPERIMENT 7

Preparing an LLM Application for Production-Ready Deployment

Lab Manual

1. Aim

To prepare the Customer Support RAG application from Experiment 5 for API-based use by adding input validation, error handling, basic logging, response-time measurement, and endpoint testing.

2. Learning Outcomes

- Explain the difference between a prototype and a production-oriented application.
- Expose an existing RAG application through a FastAPI endpoint.
- Choose either a cloud-hosted free/limited-quota LLM API or a local LLM served by Ollama.
- Validate requests, handle failures safely, and measure response time.
- Test endpoints using the interactive API documentation.

3. Background / Theory

A notebook-based RAG prototype is useful for experimentation, but an API makes it easier for other applications to send a question and receive an answer. FastAPI provides HTTP endpoints. Production-oriented preparation also includes validating inputs, handling failures, logging useful events, and measuring response time.

Keep the RAG pipeline separate from the LLM backend. The retriever finds relevant documents and builds a prompt; the backend generates the answer. Separating these components lets you change the model backend without rebuilding the whole application.

4. Required Tools

- Python 3.10 or later, VS Code or another editor, and a terminal.
- The working RAG project from Experiment 6, including its knowledge base and retrieval logic.
- FastAPI and Uvicorn, plus an HTTP client for testing.
- One model option from Section 5. Cloud inference does not require a local GPU; Ollama performance depends on the computer.

5. Choose ONE LLM Backend

Choice	Where does the model run?	Student tasks
A. Cloud-hosted API	The model runs on a provider's server; the application calls it over the internet.	Choose an available free-quota/trial API, obtain a key if required, and configure endpoint and model name.
B. Ollama local LLM	The model runs locally on the computer; the app calls a local endpoint.	Install Ollama, download a small model, start it, and configure the app to use the local endpoint.

Important: free cloud inference is normally quota-limited, may have rate limits, and can change. It is not unlimited. Never commit API keys to GitHub or show them in screenshots. Use a provider and model that are currently available.

5A. Option A — Cloud-Hosted Free/Trial API

- Choose a provider that currently offers a free quota or trial for text generation. Hugging Face Inference Providers is one place to check.
- Create an account and access token/API key if required. Keep it private.
- Record the provider's API base URL, model identifier, and request format; these vary by provider.
- Store the endpoint, model identifier, and key in environment variables or a local .env file excluded from version control.
- Check the quota before testing. If no free quota is available, use Option B or a mock backend to test the API structure.

Reference: https://huggingface.co/docs/inference-providers

https://www.kdnuggets.com/5-free-llm-api-providers-you-can-use-in-2026

5B. Option B — Ollama Local LLM

- Install Ollama from https://ollama.com and open a terminal.
- Download a small model suitable for the computer. For example:

```
ollama pull qwen2.5:0.5b
```

Run it to check that it responds:

```
ollama run qwen2.5:0.5b
```

- Ollama normally provides a local API at http://localhost:11434. The model weights are stored on the computer, separately from the FastAPI application.
- If the model is too slow or the computer runs out of memory, choose a smaller model or use Option A.

Reference: https://docs.ollama.com/

6. Procedure

- Create a project folder. Move or reuse the working RAG logic from Experiment 6 in a module such as support_rag.py. Keep document loading, chunking, embeddings, vector search, and prompt construction there.
- Create app.py with FastAPI. Add POST /ask to accept a question and call the existing RAG function. Add GET /health to report whether the API process is running.
- Initialize reusable components, such as the embedding model and vector index, once where practical; do not reload them for every request.
- Validate input: reject an empty question and set a reasonable maximum length, such as 1,000 characters.
- Add controlled error handling. Invalid input should return a client error; model/provider failures should return a generic server error. Do not expose stack traces, API keys, or private configuration.
- Measure response time in milliseconds. Log useful operational details such as success/failure and latency, but avoid logging secrets or unnecessary personal data.
- Run the app and open http://127.0.0.1:8000/docs. Submit a question through POST /ask.
- Test one question answerable from the knowledge base and one unsupported question. The application should acknowledge insufficient information instead of inventing a policy.
- Test invalid input and a backend failure. For a cloud API, check the key, quota, network, and model name. For Ollama, check that Ollama is running and the model is downloaded.
- Record results in the observation table and answer the viva questions.

7. Suggested Project Structure

Reuse Experiment 5 code rather than rebuilding the whole RAG system. Keep the files simple:

```
experiment7/
  app.py              # API endpoints and validation
  support_rag.py      # Existing RAG pipeline and chosen model backend
  requirements.txt    # Python dependencies
  .env                # Local secrets/configuration; never upload
```

8. Testing Checklist

- GET /health returns a successful status.
- A valid question receives an answer.
- An empty question is rejected.
- A very long question is rejected or safely limited.
- An unsupported question triggers an uncertainty/fallback response.
- A backend/network failure produces a controlled error.
- Response time is recorded.
- No API key or secret appears in source control, logs, or screenshots.

9. Observation Table

Test case	Expected result	Actual result	Pass/Fail
Health endpoint	API reports running		
Supported question	Grounded answer is returned		
Unsupported question	Insufficient information/fallback		
Empty question	Validation error		
Backend unavailable	Controlled error; no secret details		
Response-time measurement	Latency is recorded		

10. Result

The existing RAG application was exposed through a FastAPI endpoint and tested with input validation, error handling, basic logging, and response-time measurement.

Selected backend: ☐ Cloud-hosted API ☐ Ollama local LLM

11. Discussion Questions

- What is the purpose of FastAPI in this experiment?
- What is the difference between a local LLM and a cloud-hosted LLM API?
- Does hosting FastAPI in the cloud automatically host the LLM there?
- Why should an API key not be hard-coded in Python source?
- What is input validation, and why is a maximum question length useful?
- What is the purpose and limitation of a health endpoint?
- Why should the model and vector index not be loaded for every request?
- What should RAG do when the answer is absent from its knowledge base?
- Name two reasons a cloud inference request might fail.
- Is this lab application fully production-ready? Name two additional requirements for a real public service.

12. Precautions and Limitations

- Free inference APIs can change quotas, available models, and terms. Confirm availability before the lab.
- Do not send confidential, medical, financial, or personal data to a third-party model API.
- Ollama runs models locally, but large models may exceed RAM or disk capacity. Start with a small model.
- A real public service additionally needs authentication, HTTPS, rate limiting, monitoring, secure secret management, load testing, model evaluation, and a recovery plan.

13. References

- FastAPI: https://fastapi.tiangolo.com/
- Uvicorn: https://www.uvicorn.org/
- Ollama: https://docs.ollama.com/
- Hugging Face Inference Providers: https://huggingface.co/docs/inference-providers
- LangChain: https://python.langchain.com/docs/